

06_checklist

10. Production-Readiness Checklist

A tickable gate before calling the dbt layer production-grade. Group it into the CI pipeline (`dbt build` + tests + `dbt_project_evaluator` + the eval gate) so "production-ready" is *enforced*, not aspirational.

Project & configuration

- ☐ `require-dbt-version: "≥1.7.0"` pinned; `packages.yml` versions pinned and `dbt deps` reproducible.
- ☐ `dbt_project.yml` sets folder-level materializations, `partition_by` `event_date`, `cluster_by`, `insert_overwrite`, `persist_docs`, `+group` / `+access`, `+tags`.
- ☐ `profiles.yml`: `dev` → `helios_dev`, `prod` → `helios_prod`; prod auth via Workload Identity Federation (no JSON keys); `maximum_bytes_billed` set per target.
- ☐ `vars` centralize the build window, incremental lookback (3 days), engagement threshold (10000 ms).

Sources

- ☐ All raw access via `source()` — no model refs a raw table directly.
- ☐ `source freshness` configured (warn 36h / error 48h) and gates the build in prod; informational-only on the static sample.
- ☐ Date-sharded wildcard pruned via `_TABLE_SUFFIX`; build window bounded by vars.

Staging

- ☐ One staging model per source entity; views; rename/cast/light-flatten only — no joins/aggregations/ `SELECT *`.
- ☐ `get_event_param`, `sessionize`, `channel_group_case` macros are the single source of those transforms.
- ☐ Keys tested `unique` + `not_null`.

Intermediate (keystones)

- ☐ `int_ga4__sessionized` and `int_ga4__funnel_steps` have **golden-value unit tests** (these fail silently).
- ☐ Funnel monotonicity holds (`assert_funnel_monotonicity`); `session_key` unique per session.
- ☐ `traffic_source` first-touch fallback documented and tested.

Marts

- ☐ Every mart has a single declared grain + primary-key uniqueness test.
- ☐ Conformed dims (`dim_date` , `dim_channels` , `dim_users` , `dim_items`) shared across facts; `relationships` tests pass.
- ☐ `dim_channels` `accepted_values` = exactly the 10 channel groups.
- ☐ Marts are wide; the 5 semantic grains (`fct_funnel` , `fct_sessions` , `fct_orders` , `fct_order_items` , `fct_cohorts`) exist and reconcile.
- ☐ Incremental facts re-materialize the 3-day lookback correctly on re-run (idempotent).

Testing

- ☐ `dbt build` (not separate `run` + `test`); generic + singular + custom-generic + unit tests present.
- ☐ `revenue_reconciles` to the cent; `session_conversion_rate` bounded [0,1].
- ☐ Reconciliation to `warehouse-mcp.reconcile` within 0.5%.
- ☐ The 50-scenario eval benchmark runs as the CI integration test; gate: root-cause $\geq 85\%$, hallucination = 0.
- ☐ `store_failures` enabled; severities (warn vs error) deliberate.

CI/CD

- ☐ Slim CI on PRs: `state:modified+` with `--defer` to the prod manifest; full build on merge to `main` .
- ☐ `dbt_project_evaluator` enforces: every model documented, tested, has a contract, correct layer refs.
- ☐ Prod build scheduled after source freshness passes.

Freshness & cost

- ☐ Per-layer freshness SLAs defined; stale source blocks the run + alerts (prod).
- ☐ `maximum_bytes_billed` caps every job; partition pruning verified; no `SELECT *` in built models.
- ☐ Per-run byte budget (≤ 5 GiB) honored end-to-end.

Lineage & governance

- ☐ `exposures.yml` declares the 7 agents + the Decision Brief; `manifest.json` / `catalog.json` published.
- ☐ Model `groups` + `access` (staging/intermediate private, marts public) enforced; mart `contracts` enforce schemas for downstream consumers.
- ☐ Lineage traces a metric → mart → raw GA4 column (the "0 hallucinated columns" chain).

Documentation

- ☐ Every model + column has a description; doc blocks for reusable definitions; `persist_docs` pushes to BigQuery metadata.
- ☐ `dbt docs` site generated in CI; model `meta` carries owner/maturity/contains_pii.
- ☐ Docs kept in lockstep with the Bible / CLAUDE.md / `semantic_layer.yaml`.

Production-frontier (Bible Phase 3+, deferred)

- ☐ Multi-tenant isolation (per-tenant datasets + byte budgets); warehouse-agnostic adapters behind the semantic layer.
- ☐ Intraday/streaming ingestion (replaces the static daily sample); cross-project lineage via dbt Mesh.